

Certified Finite-State Induction for a Perturbed Hofstadter Recursion

Marco Mantovanelli
Independent Researcher
marco@mantovanelli.de

Abstract

We study the parity-perturbed Hofstadter-type recursion

$$Q(1) = Q(2) = 1, \quad Q(n) = Q(n - Q(n - 1)) + Q(n - Q(n - 2)) + (-1)^n.$$

We provide a finite machine-checkable certificate and verifier suite which, when checked, yields a finite-state induction proof that the recursion is well-defined for all $n \geq 1$.

The proof extracts a finite symbolic recursive model from a directly verified initial trace and then verifies an exported machine-readable certificate by independent checkers. The certificate consists of finite symbolic word systems, radius- R contexts, context-extension records, symbolic realizations, and arithmetic recurrence records.

The checkers verify symbolic closure, cycle factorization, faithfulness to an independently recomputed trace, arithmetic correctness, parity consistency, and strict backwardness of all certified recursive dependencies. The length of the computed trace is not used as evidence for global well-definedness; instead, exhaustiveness is certified over the declared finite symbolic transition system.

Combining these finite checks with a minimal-counterexample induction rules out a first undefined recursive call.

Keywords: meta-Fibonacci recursion; Hofstadter recurrence; well-definedness; nested recurrence; computer-assisted proof; finite-state induction; symbolic dynamics; renormalization; certificate verification; recursive dependency graph.

MSC2020: Primary 11B37, 68Q45; Secondary 37B10, 03D25, 68Q60, 68W30.

Contents

1	Introduction	3
2	Symbolic Recursive Model	5
2.1	Symbolic words	6
2.2	Extraction by return words	7
2.3	Finite symbolic alphabet	7
2.4	Stable prefix	8
2.5	Cyclic symbolic closure	8
2.6	Role of the symbolic model	9
3	Certified Recursive Composition	9
3.1	Symbol closure	10
3.2	Factorization closure	11
3.3	Why one-step closure is not enough	11
3.4	Interpretation	11

4	Certified Backward Induction	11
4.1	Recursive realizations	12
4.2	Backwardness	13
4.3	Arithmetic correctness	13
4.4	Certificate statistics	14
4.5	Inductive consequence	14
4.6	Role in the global proof	15
5	Finite-State Induction Principle	15
5.1	Why the induction is finite	16
5.2	Induction as a well-founded dependency graph	16
5.3	Role of the cyclic symbolic system	17
5.4	Consequence	17
6	Certificate Format and Realization Map	17
7	Coverage and Anchoring	23
8	Main Theorem	24
8.1	Logical dependencies of the proof	26
8.2	Certificate summary	26
8.3	Conclusion	27
9	Computational Verification	27
9.1	Role of the computational component	27
9.2	Certificate format	27
9.3	Symbol closure checker	28
9.4	Pattern realization checker	29
9.5	Factorization checker	29
9.6	Coverage and anchoring checker	29
9.7	Backward induction checker	30
9.8	Faithfulness checker	31
9.9	Verification guarantees	31
9.10	Trusted code base	31
9.11	Logical status of the programs	32
9.12	Reproducibility	32
9.13	Summary	33
10	Scope and Interpretation	33
10.1	What is proved	33
10.2	What is not claimed	34
10.3	Role of computation	34
10.4	Interpretation of the finite symbolic model	34
10.5	Why the result is stronger than numerical verification	35
10.6	Limitations	35
10.7	Conceptual significance	35
10.8	Summary	35
11	Future Directions	36
11.1	Formal verification	36
11.2	Reducing the trusted code base	36
11.3	Asymptotic behavior	37
11.4	Extension to other meta-Fibonacci recursions	37

11.5 Symbolic renormalization	37
11.6 Recursive automata for nested recurrences	38
11.7 Automated model extraction	38
11.8 Comparison with the classical Hofstadter sequence	38
11.9 Conclusion	38

1 Introduction

Meta-Fibonacci recursions are recursive sequences in which the recursive arguments themselves depend on previously computed values. Among the best known examples is Hofstadter’s Q -sequence, introduced in [13]:

$$Q(n) = Q(n - Q(n - 1)) + Q(n - Q(n - 2)),$$

with initial conditions

$$Q(1) = Q(2) = 1.$$

Despite its deceptively simple form, the sequence exhibits highly irregular behavior and remains poorly understood. Fundamental questions concerning growth, regularity, decidability, and global well-definedness are still open. The sequence has therefore become a central object in the theory of nested and meta-Fibonacci recursions.

A substantial literature studies variants and relatives of the Hofstadter recursion. Tanny introduced one of the first systematically analyzable relatives in [19], showing that certain modified recursions exhibit surprisingly regular asymptotic behavior. Pinn investigated the coexistence of order and apparent chaos in the classical Q -sequence [18]. Subsequent work explored broader families of Hofstadter-type recursions, including generational structures, linear recurrent solutions, and experimentally observed regularities [8, 2, 1, 9, 10, 11].

At the same time, several works demonstrated the complexity and potential pathological behavior of nested recursions. Undecidability phenomena for nested recurrence relations were established by Celaya and Ruskey in [6], illustrating that even apparently elementary recursive systems may possess enormous computational complexity. Further investigations of the arithmetic and dynamical properties of meta-Fibonacci systems appear in [22, 20, 21, 3].

The present work studies the perturbed Hofstadter-type recursion

$$Q(1) = Q(2) = 1,$$

$$Q(n) = Q(n - Q(n - 1)) + Q(n - Q(n - 2)) + (-1)^n.$$

This sequence is listed in the OEIS as sequence A394051 [14]. Preliminary experimental observations suggesting a dyadic self-similar structure were reported in [16]. Cloitre developed a complementary analytic and combinatorial approach to the same perturbed recursion, which proved well-definedness and the convergence estimate

$$\left| Q(n)/n - \frac{1}{2} \right| = O\left(1/\sqrt{\log n}\right)$$

using arch decompositions, interleaving transducers, and Catalan-type frequency laws [7]. The present work is complementary to Cloitre’s analytic and combinatorial approach. Its objective is an independent certificate-based proof of global well-definedness, rather than an asymptotic analysis.

The central question addressed here is:

Does the recursion remain valid for all $n \geq 1$, i.e. do all recursive arguments remain positive indefinitely?

For classical Hofstadter-type recursions, such questions are often extremely difficult because recursive dependencies may propagate in highly nonlocal and irregular ways. Traditional analytical approaches appear insufficient to control the full recursive dependency graph.

The approach developed here is instead based on a finite symbolic renormalization framework. A verified trace is used to extract candidate symbolic systems. The proof then uses only the exported finite certificates and the independent verification programs applied to them. Iterated extraction reveals a hierarchy of recursively stable symbolic systems:

$$S \rightarrow T \rightarrow U \rightarrow V.$$

The trace is used only to propose the finite state space. Exhaustiveness is not inferred from the length of the trace. It is certified by checking that the transition relation generated by the recurrence maps every admissible certified state back into the same finite state space.

The central phenomenon observed experimentally is that these symbolic systems exhibit stable recursive self-similarity under composition. The main contribution of this paper is to convert this phenomenon into a finite machine-checkable certificate and a corresponding finite-state induction proof.

The proof combines several ingredients:

1. extraction of finite symbolic recursive systems from verified traces,
2. certification of recursive composition closure,
3. exhaustive verification of symbolic factorization properties,
4. certification that every recursive dependency is strictly backward,
5. finite-state induction over the resulting symbolic automaton.

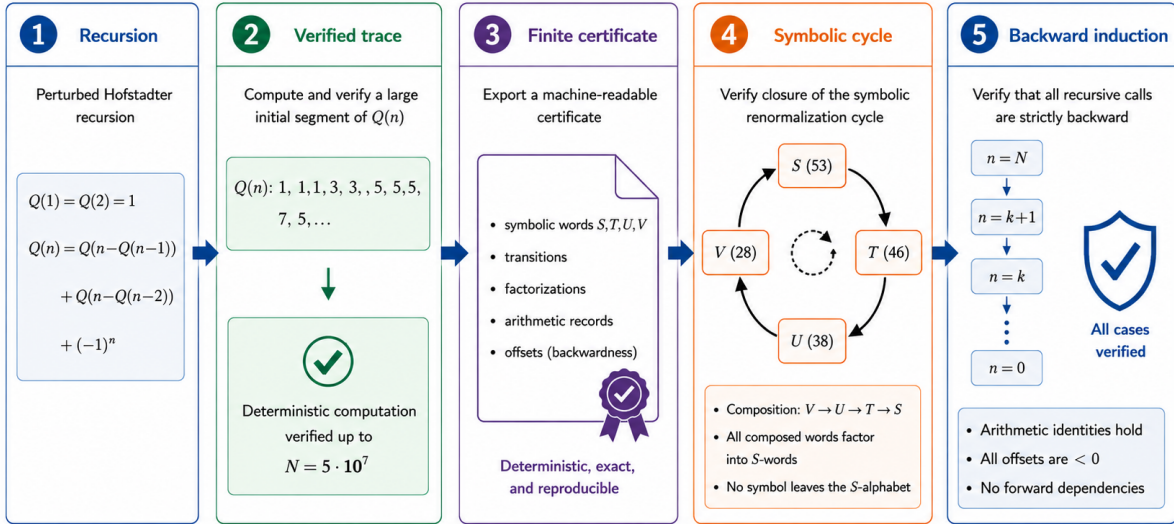


Figure 1: Verification pipeline for the certified finite-state induction proof. The trace consists of the verified initial computation of the sequence together with the recursive dependency data used to construct the finite certificate. The certificate is then checked for symbolic closure, cycle factorization, arithmetic consistency, and strict backwardness.

Figure 1 summarizes the overall verification pipeline. The finite computation is used to produce a machine-checkable certificate, while the final proof uses only the verified finite symbolic cycle and the backward induction certificate.

The resulting argument is fundamentally computational, but unlike purely heuristic experimental mathematics, every step is reduced to finite certified verification problems. All symbolic certificates are explicit, and all verification programs are deterministic and reproducible.

Conceptually, the proof is related to ideas from symbolic dynamics [15], recursive automata, and finite constraint systems. The recursive closure properties verified here resemble finite consistency mechanisms occurring in symbolic dynamical systems and constraint propagation frameworks [4, 12]. The work is also loosely connected to recent investigations of meta-automatic and recursively generated symbolic structures [5].

Our main result is the following.

Theorem 1.1. *The perturbed Hofstadter recursion*

$$Q(n) = Q(n - Q(n - 1)) + Q(n - Q(n - 2)) + (-1)^n$$

with initial conditions

$$Q(1) = Q(2) = 1$$

is well-defined for all $n \geq 1$.

The proof is given in Section 8.

More precisely, we construct a finite recursively closed symbolic system whose certified induction structure guarantees that every recursive call ultimately terminates at previously verified indices.

The proof does not attempt to classify all behaviors of the recursion, nor does it provide a closed-form description of the sequence. Instead, the paper establishes that the recursive dependency structure admits a finite-state certification sufficient to prove global well-definedness.

Finally, we emphasize that the computational framework developed here is potentially more general than the particular sequence studied. The extraction of recursively closed symbolic systems may provide a new approach for analyzing broader classes of nested recursions whose direct analytic treatment appears inaccessible.

2 Symbolic Recursive Model

The proof is based on a finite symbolic model extracted from verified finite traces of the recursion. The purpose of this model is not to give a closed form for $Q(n)$, but to encode the recursive dependency structure in a finite and machine-checkable form.

In the original recurrence,

$$Q(n) = Q(n - Q(n - 1)) + Q(n - Q(n - 2)) + (-1)^n,$$

the two recursive arguments depend on the previous values of Q . Thus, a direct symbolic analysis of the sequence values is difficult. Instead, we encode finite local recursion patterns and study how these patterns are composed under repeated recursive expansion.

The symbolic model consists of finite collections of words

$$S, \quad T, \quad U, \quad V,$$

together with transition sequences and factorization rules between them. These objects are extracted from a verified trace of the recursion and then checked independently by finite verification programs.

2.1 Symbolic words

A symbolic word is a finite word in an integer alphabet. It represents a certified local recursion pattern, i.e. a finite symbolic description of how a block of recursive dependencies is organized.

At the four relevant levels of the renormalization hierarchy, we obtain finite families of symbolic words

$$S_i, \quad T_i, \quad U_i, \quad V_i.$$

Here S_i denotes a word at the S -level, T_i a word at the T -level, and similarly for U_i and V_i .

The certified extraction gives:

$$|S| = 53, \quad |T| = 46, \quad |U| = 38, \quad |V| = 28.$$

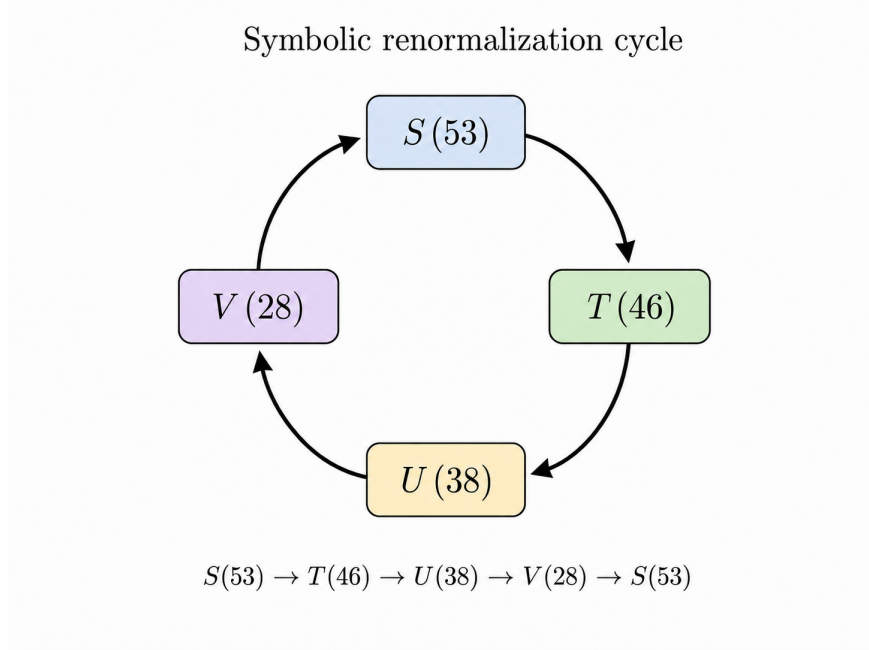


Figure 2: Symbolic renormalization cycle of the certified word systems. The cycle $S(53) \rightarrow T(46) \rightarrow U(38) \rightarrow V(28) \rightarrow S(53)$ is not a one-step closed substitution system. Closure is obtained after one full cycle, followed by factorization of the composed words into certified S -words.

Figure 2 displays the symbolic renormalization cycle used throughout the proof. The cyclic structure is the central symbolic object: the smaller stable V -core does not close under one substitution step, but its expansion through U , T , and S returns to the certified S -system after factorization.

Thus, the four word systems are

$$S = \{S_0, \dots, S_{52}\},$$

$$T = \{T_0, \dots, T_{45}\},$$

$$U = \{U_0, \dots, U_{37}\},$$

and

$$V = \{V_0, \dots, V_{27}\}.$$

Each word at one level is written in the alphabet of the preceding level. For example, a T -word is a finite word in S -symbols, a U -word is a finite word in T -symbols, and a V -word is a finite word in U -symbols.

Concretely, a word such as

$$T_i = S_{a_1} S_{a_2} \cdots S_{a_k}$$

means that the i -th symbolic block at the T -level factors into the S -level blocks

$$S_{a_1}, S_{a_2}, \dots, S_{a_k}.$$

The same convention applies recursively:

$$U_i = T_{b_1} T_{b_2} \cdots T_{b_\ell},$$

and

$$V_i = U_{c_1} U_{c_2} \cdots U_{c_m}.$$

The words should be understood as symbolic recursion blocks. Their role is to encode how local recursive structures reappear after renormalizing the trace.

2.2 Extraction by return words

The symbolic words are extracted using return-word decompositions. Given a symbolic sequence at one level, a marker pair is chosen, and the blocks between consecutive occurrences of this marker are recorded.

For the certified system used here, the relevant marker is stable across successive levels:

$$(1, 1).$$

At the first symbolic level, this corresponds to the marker pair

$$(R_0, R_0),$$

and at later levels to

$$(M_1, M_1), \quad (N_1, N_1), \quad (P_1, P_1),$$

and so on.

This return-word construction produces the successive symbolic systems

$$S, \quad T, \quad U, \quad V.$$

The important point is that this construction is not used as a heuristic in the proof. Once the symbolic systems have been extracted, all subsequent claims are verified by independent checkers operating only on the explicit finite word data.

2.3 Finite symbolic alphabet

All final recursive realizations in the certified system are represented over the finite alphabet

$$\Sigma = \{0, 1, \dots, 52\}.$$

This alphabet is identified with the S -level word indices:

$$\Sigma = \{S_0, S_1, \dots, S_{52}\}.$$

The larger symbolic levels are therefore interpreted as higher-order words over this base alphabet after expansion. In particular, a V -word can be expanded successively as

$$V \longrightarrow U \longrightarrow T \longrightarrow S.$$

After this three-step expansion, the result is a finite word over

$$S_0, \dots, S_{52}.$$

The key certified property is that no expansion of the stable core escapes this alphabet. More precisely, for every $0 \leq i < 28$, the four-step symbolic expansion of V_i lands entirely inside the S -alphabet:

$$V_i \mapsto \text{a finite word over } \{S_0, \dots, S_{52}\}.$$

This is the first closure property of the finite symbolic model.

2.4 Stable prefix

The systems S and V share a stable prefix of length 28. That is, the first 28 words of the V -system have the same index patterns as the first 28 words of the S -system:

$$V_i \equiv S_i \quad (0 \leq i < 28),$$

where equivalence means equality of the symbolic word pattern after renaming the ambient level.

This was verified by the prefix checker:

$$V_0, \dots, V_{27} = S_0, \dots, S_{27}.$$

This stable prefix is not, by itself, closed under one-step substitution. Indeed, some of the words $V_0, \dots, V_{27}$ use symbols beyond V_{27} . However, the crucial fact is that the full four-level composition

$$V \rightarrow U \rightarrow T \rightarrow S$$

is closed after factorization into certified S -words.

Thus, the stable object is not a one-level substitution system, but a finite cyclic symbolic system.

2.5 Cyclic symbolic closure

The renormalization cycle has the form

$$S \longrightarrow T \longrightarrow U \longrightarrow V \longrightarrow S.$$

The word counts along the cycle are

$$53 \longrightarrow 46 \longrightarrow 38 \longrightarrow 28 \longrightarrow 53.$$

This should not be interpreted as a monotone reduction of alphabets. Rather, the smaller V -level stable core expands under composition back into the larger S -level alphabet.

The certified closure statement is:

For every $0 \leq i < 28$, the composed expansion of V_i through U , T , and S factors completely into certified S -words.

Equivalently, each composed word

$$\Phi(V_i)$$

admits a factorization

$$\Phi(V_i) = S_{j_1} S_{j_2} \cdots S_{j_r}, \quad 0 \leq j_k \leq 52.$$

This factorization property is the central symbolic closure condition used in the finite induction proof.

2.6 Role of the symbolic model

The symbolic model provides a finite replacement for the infinite recursive dependency structure.

Instead of attempting to analyze all recursive calls directly, we verify that:

1. every certified symbolic word expands into certified symbolic words,
2. recursive composition remains inside the finite symbolic alphabet,
3. the stable prefix returns after one full renormalization cycle,
4. and the resulting symbolic expansions can be factored completely into certified S -words.

Together with the arithmetic and backwardness checks described later, this symbolic closure is what makes the finite-state induction possible.

The remainder of the proof therefore reduces the well-definedness of the recursion to two finite verification tasks:

1. symbolic closure under cyclic composition;
2. strict backwardness of every certified recursive dependency.

3 Certified Recursive Composition

The first major verification step establishes that the recursive composition does not leave the finite symbolic universe constructed in the previous section.

Recall that the symbolic hierarchy has the form

$$S \longrightarrow T \longrightarrow U \longrightarrow V \longrightarrow S.$$

A word at one level is written on the alphabet of the preceding level. Thus, a V -word is initially a word in U -symbols. By substituting the corresponding U -words, then the corresponding T -words, and finally the corresponding S -words, one obtains a finite word over the S -alphabet.

The essential closure property is that this process never produces an uncontrolled or undefined symbolic object.

Definition 3.1 (Four-step composition). *Let V_i be one of the certified V -level words. The four-step composition of V_i , denoted by*

$$\Phi(V_i),$$

is obtained by the successive expansion

$$V_i \longrightarrow U \longrightarrow T \longrightarrow S.$$

Equivalently, if

$$V_i = U_{a_1} U_{a_2} \cdots U_{a_r},$$

then each U_{a_j} is expanded into T -words, each resulting T -word is expanded into S -words, and the final result is a finite word over the alphabet

$$\{S_0, \dots, S_{52}\}.$$

The central certified statement is the following.

Theorem 3.2 (Composition Closure). *For every $0 \leq i < 28$, the four-step composition*

$$\Phi(V_i)$$

is a finite word on the certified S -alphabet and completely factors as a concatenation of certified S -words.

More explicitly, for every $0 \leq i < 28$, there exist indices

$$j_1, \dots, j_r \in \{0, \dots, 52\}$$

such that

$$\Phi(V_i) = S_{j_1} S_{j_2} \cdots S_{j_r}.$$

Thus the composed symbolic realization contains no symbol outside the certified alphabet and admits a complete factorization into previously certified S -level blocks.

This theorem is verified by the program

`cycle_composition_factor_checker.c.`

The checker performs the following finite operations:

1. it loads the certified word systems S, T, U, V ;
2. for each $0 \leq i < 28$, it expands V_i through the chain

$$V \rightarrow U \rightarrow T \rightarrow S;$$

3. it checks that every symbol in the resulting word lies in

$$\{0, \dots, 52\};$$

4. it factors the resulting word as a concatenation of certified S -words;
5. it reports failure if any symbol is outside the certified alphabet or if any expansion cannot be factored.

The verified output is:

`checked_words = 28,`
`factored = 28,`
`failed = 0,`
`symbol_bad_total = 0.`

Hence every member of the stable V -prefix survives one complete renormalization cycle and returns to the certified S -system after factorization.

3.1 Symbol closure

The first part of the verification checks symbol closure.

For every $0 \leq i < 28$, the expanded word $\Phi(V_i)$ contains only symbols in the range

$$0, \dots, 52.$$

Equivalently,

$$\Phi(V_i) \in \{S_0, \dots, S_{52}\}^*.$$

This excludes the possibility that the recursive composition creates a new symbolic state that is not already present in the certified system.

In the verification output, this is recorded as

`symbol_bad_total = 0.`

3.2 Factorization closure

Symbol closure alone is not sufficient. It only states that the final expanded word uses allowed symbols. For induction, we need the stronger property that the word can be parsed into certified recursion blocks.

The checker therefore verifies that every $\Phi(V_i)$ admits a complete factorization

$$\Phi(V_i) = S_{j_1} S_{j_2} \cdots S_{j_r}.$$

The number of factors may be large and depends on i . For example, the verification produced factor counts, such as

$$127, \quad 247, \quad 848, \quad 2510, \quad 4096,$$

for different V_i . The exact factorization is not used as a heuristic; it is checked explicitly by matching it to the finite list of certified S -words.

The important conclusion is:

$$\text{factored} = 28, \quad \text{failed} = 0.$$

Thus, all 28 stable V -words return, after one full symbolic cycle, to concatenations of certified S -words.

3.3 Why one-step closure is not enough

It is important to emphasize that the stable core is not closed after a single symbolic substitution.

For example, the stable prefix

$$V_0, \dots, V_{27}$$

may contain references to symbols beyond the range $0, \dots, 27$. Thus, the 28-word prefix is not a closed one-level substitution system.

The correct closure statement is cyclic rather than one-step:

$$V \longrightarrow U \longrightarrow T \longrightarrow S \longrightarrow \text{factorization into } S\text{-words}.$$

This is why the proof uses a finite renormalization cycle rather than a single finite automaton with a closed substitution rule.

3.4 Interpretation

The composition closure theorem shows that the symbolic recursion system is internally stable under recursive propagation.

Starting from any certified stable V -word, one full recursive renormalization cycle produces a word that can again be decomposed into certified S -words. Therefore recursive expansion does not create new symbolic types, does not leave the certified alphabet, and does not generate an unfactorable remainder.

In this sense, the finite symbolic system behaves as a recursively stable automaton.

This closure property is one of the two essential ingredients in the proof of well-definedness. The second ingredient, established in the next section, is that every certified recursive dependency is strictly backward.

4 Certified Backward Induction

The second major component of the proof is the backward induction verification. Symbolic closure alone shows that recursive composition remains inside a finite certified system, but it does not by itself imply well-definedness of the original recursion. For well-definedness one must also verify that every recursive dependency points strictly to an earlier index.

This section describes the finite certificate that establishes precisely this property.

4.1 Recursive realizations

The verification certificate contains a finite list of recursive realizations. Each realization corresponds to a certified occurrence of the recurrence relation inside the symbolic model.

Each arithmetic record has the form

$$(t, q_t, r_a, o_a, \sigma_a, q_a, r_b, o_b, \sigma_b, q_b, \varepsilon),$$

where:

- t is the local target offset relative to the block anchor L ,
- q_t is the value assigned to the target,
- r_a and r_b are relative offsets of the two recursive arguments,
- o_a, o_b are auxiliary offsets recorded by the exporter,
- σ_a, σ_b are symbolic labels of the recursive argument positions,
- q_a, q_b are the certified values at the two recursive arguments,
- and $\varepsilon \in \{-1, 1\}$ is the perturbation term.

The record certifies the local identity

$$q_t = q_a + q_b + \varepsilon.$$

It also certifies that the two recursive calls are located at relative positions

$$t + r_a, \quad t + r_b.$$

The verification system checks four conditions for every such record:

1. arithmetic correctness:

$$q_t = q_a + q_b + \varepsilon;$$

2. parity consistency:

$$\varepsilon = (-1)^{L+t},$$

where L is the global block anchor of the realization and t is the local target offset of the arithmetic record;

3. strict backwardness of the first recursive call:

$$r_a < 0;$$

4. strict backwardness of the second recursive call:

$$r_b < 0.$$

Thus each certified recurrence instance is verified arithmetically, parity-consistently, and inductively.

The checker processes the full exported certificate and verifies:

$$4,735,941$$

recursive realizations and

$$49,999,571$$

arithmetic recursion records.

The verification output reports:

$$\begin{aligned} \text{realizations_checked} &= 4,735,941, \\ \text{arithmetic_records_checked} &= 49,999,571. \end{aligned}$$

4.2 Backwardness

The crucial inductive property is that no certified recursive call ever points forward or sideways.

Theorem 4.1 (Strict Backwardness). *Every certified recursive dependency is strictly backward.*

Equivalently, if a certified record represents a recurrence instance at a target position t , then both recursive arguments satisfy

$$t + r_a < t, \quad t + r_b < t.$$

Since this is equivalent to

$$r_a < 0, \quad r_b < 0,$$

the verification reduces to checking the signs of the two relative offsets in every arithmetic record.

The complete verification gives:

$$\text{min_rel_seen} = -2441147,$$

$$\text{max_rel_seen} = -10.$$

In particular,

$$r_a \leq -10, \quad r_b \leq -10$$

for every certified recursive call. Hence every recursive dependency is not only backward, but uniformly separated from the target by at least ten positions.

The checker reports:

$$\text{non_backward_bad} = 0.$$

Thus no non-backward recursive dependency occurs anywhere in the certificate.

Proof. The program `well_definedness_induction_checker.c` scans every arithmetic record in the exported certificate. For each record it reads the relative offsets r_a and r_b . It then verifies

$$r_a < 0, \quad r_b < 0.$$

If either inequality fails, the checker reports a non-backward recursion failure. Since the complete scan reports

$$\text{non_backward_bad} = 0,$$

all certified recursive dependencies are strictly backward. □

4.3 Arithmetic correctness

The backwardness condition guarantees inductive descent. We must also verify that the certified recurrence identities are arithmetically correct.

Theorem 4.2 (Arithmetic Consistency). *Every certified arithmetic recursion identity holds exactly.*

That is, for every arithmetic record in the certificate,

$$q_t = q_a + q_b + \varepsilon.$$

The complete verification reports:

$$\text{arithmetic_bad} = 0.$$

Therefore every exported recurrence identity agrees exactly with the defining perturbed recursion.

Proof. For each arithmetic record, the checker computes

$$q_a + q_b + \varepsilon$$

and compares it with the certified value q_t . Any mismatch is reported as an arithmetic failure. Since the scan of all

$$49,999,571$$

records produces no arithmetic failures, every certified identity is arithmetically valid. $\square$

4.4 Certificate statistics

The full induction certificate produced the following verification statistics:

$$\begin{aligned} \text{radius} &= 20, \\ \text{words} &= 13, \\ \text{contexts_seen} &= 2923, \\ \text{realizations_header} &= 4,735,941, \\ \text{realizations_checked} &= 4,735,941, \\ \text{arithmetic_records_checked} &= 49,999,571. \end{aligned}$$

The induction statistics were:

$$\begin{aligned} \text{min_rel_seen} &= -2441147, \\ \text{max_rel_seen} &= -10, \\ \text{arithmetic_bad} &= 0, \\ \text{non_backward_bad} &= 0, \\ \text{parity_bad} &= 0, \quad \text{malformed_bad} = 0. \end{aligned}$$

Thus the certificate is complete with respect to its header count, all arithmetic records are well-formed, all recurrence identities are correct, and all recursive calls are strictly backward.

4.5 Inductive consequence

The strict backwardness property is the finite analogue of ordinary strong induction.

Suppose a certified recurrence instance occurs at position t . The two recursive calls occur at positions

$$t + r_a, \quad t + r_b,$$

with

$$r_a < 0, \quad r_b < 0.$$

Therefore both recursive arguments are smaller than t . If all positions below t are already known to be defined, then the certified recurrence instance at t is also defined.

Combined with arithmetic correctness, this gives the following finite induction principle.

Proposition 4.3 (Certified Backward Induction Step). *Every certified recurrence instance is valid assuming all earlier certified instances are valid.*

Proof. Let a certified recurrence instance be given at local target position t . By strict backwardness, both recursive arguments occur at positions strictly smaller than t . By the induction hypothesis, the corresponding values are already defined. By arithmetic consistency, the target value is exactly the sum of these two earlier values plus the perturbation term. Hence the recurrence instance at t is valid. $\square$

4.6 Role in the global proof

The symbolic closure theorem from the previous section guarantees that recursive expansion stays inside the certified symbolic universe. The backward induction verification guarantees that every certified recursive dependency descends to earlier positions.

Together, these two facts rule out the only possible mechanism by which the recursion could fail: a recursive call to an undefined or nonpositive future-dependent argument.

Thus the well-definedness proof rests on the combination of:

$$\text{symbolic closure} \quad + \quad \text{arithmetic and parity correctness} \quad + \quad \text{strict backwardness}.$$

5 Finite-State Induction Principle

We now formulate the finite induction principle that connects the symbolic closure verification with the backward arithmetic verification.

The ordinary induction argument for a recursively defined sequence is straightforward when every recursive call is visibly made to a smaller index. In the present setting, the recursive arguments

$$n - Q(n - 1), \quad n - Q(n - 2)$$

are themselves generated by the evolving sequence, and therefore their validity is not immediate from the definition.

The certified symbolic system resolves this difficulty by replacing the infinite recursive dependency structure with a finite, recursively closed symbolic model whose dependencies have all been checked to be strictly backward.

Definition 5.1 (Certified symbolic realization). *A certified symbolic realization is a finite symbolic object occurring in the verified certificate and satisfying the following five properties:*

1. symbolic closure: *every symbolic expansion remains inside the certified symbolic alphabet;*
2. arithmetic consistency: *every certified recurrence identity satisfies*

$$q_t = q_a + q_b + \varepsilon;$$

3. parity consistency: *the perturbation term satisfies*

$$\varepsilon = (-1)^n$$

for the corresponding global target index;

4. strict backwardness: *every recursive dependency points to a strictly earlier position;*
5. recursive factorization: *every composed symbolic expansion factors into certified symbolic words.*

The concrete certificate format underlying this notion is specified in Section 6.

The previous two sections establish these properties by finite verification. Composition closure and factorization were verified by the cycle-composition factor checker, while arithmetic consistency and strict backwardness were verified by the induction checker.

The finite induction principle can now be stated as follows.

Theorem 5.2 (Certified Finite-State Induction). *Every certified symbolic realization generates only recursively valid descendants.*

Proof. Let X be a certified symbolic realization. We consider any recursive descendant produced by expanding X .

By symbolic closure, every symbolic expansion of X remains inside the certified symbolic universe. Thus no expansion step creates an unknown symbolic type.

By recursive factorization, every composed expansion can be decomposed as a concatenation of certified symbolic words. Therefore every descendant produced by recursive propagation is again represented by objects already present in the finite certificate.

It remains to verify that these descendants are valid as recurrence instances. For each certified recurrence record, arithmetic consistency gives

$$q_t = q_a + q_b + \varepsilon.$$

By parity consistency, the perturbation term in this identity is exactly

$$\varepsilon = (-1)^n$$

for the corresponding global target index. Hence, the certified identity is the defining perturbed Hofstadter recurrence, not merely an abstract local arithmetic relation.

Finally, strict backwardness gives

$$r_a < 0, \quad r_b < 0.$$

Therefore the two recursive arguments occur at positions strictly earlier than the target position. Consequently, if all earlier certified positions are valid, then the current position is valid.

This is precisely the strong induction step. Since every recursive descendant remains inside the finite certified system and every certified recursive call is strictly backward, recursive propagation cannot reach an undefined descendant.

Thus every certified symbolic realization generates only recursively valid descendants. $\square$

5.1 Why the induction is finite

The induction above is finite-state because it does not require infinitely many independent cases. All possible recursive local structures that occur in the certified system are represented by finitely many symbolic words.

The relevant finite data are:

$$|S| = 53, \quad |T| = 46, \quad |U| = 38, \quad |V| = 28.$$

The four-step composition verification checks the stable core

$$V_0, \dots, V_{27}$$

and confirms that every such word expands and factors into the certified S -system.

The backward induction checker then verifies every certified arithmetic realization in the exported certificate. Since the symbolic system is closed under composition, these finite checks cover all descendants generated by the certified symbolic dynamics.

5.2 Induction as a well-founded dependency graph

Equivalently, the certificate defines a finite family of recursive dependency patterns. Each pattern contains directed dependency edges from a target position to its recursive arguments.

The backwardness verification proves that every such edge decreases the local position. Thus the dependency graph induced by any certified expansion is well-founded.

There can be no infinite descending obstruction inside a finite interval, and there can be no forward dependency because all certified offsets are negative. Hence recursive evaluation can always be reduced to earlier values.

This is the finite-state analogue of the usual proof that a recurrence is well-defined by induction on n .

5.3 Role of the cyclic symbolic system

The symbolic model is cyclic rather than closed at a single level:

$$S \rightarrow T \rightarrow U \rightarrow V \rightarrow S.$$

This means that one should not expect the stable V -prefix alone to be closed under one substitution. Instead, closure is achieved after one complete cycle, followed by factorization into S -words.

The induction principle therefore uses the entire cycle. A recursive expansion may pass through the levels V, U, T, S , but the factorization checker verifies that after completing the cycle the expansion returns to certified S -words.

Thus, the cycle does not create an infinite hierarchy of new symbols. It is finite, closed under composition, and compatible with backward recursive evaluation.

5.4 Consequence

Combining the finite-state induction theorem with the construction of the symbolic model gives the main mechanism of the proof:

Every recursive configuration represented by the certified symbolic system expands only into earlier certified configurations, and every such expansion is arithmetically correct.

This is the formal reason why the certified system cannot produce an undefined recursive call.

6 Certificate Format and Realization Map

This section specifies the finite certificate format used to connect the actual recurrence with the symbolic induction system. The purpose of the certificate is to separate two logically distinct tasks:

1. anchoring the symbolic system in a directly verified trace of the actual recursion;
2. proving that the certified symbolic transition relation is closed under recursive propagation.

The trace is used only to propose and anchor the finite symbolic state space. Exhaustiveness is not inferred from the length of the trace. It is certified by checking that the transition relation generated by the recurrence maps every admissible certified state back into the same finite symbolic state space.

Definition 6.1 (Real recursive data at a target). *For a target index n , the real recursive data needed to evaluate $Q(n)$ are*

$$(Q(n-1), Q(n-2), n-Q(n-1), n-Q(n-2), (-1)^n),$$

together with the two dependency edges

$$n \rightarrow n-Q(n-1), \quad n \rightarrow n-Q(n-2),$$

whenever the displayed previous values are defined.

Definition 6.2 (Certified symbolic context). *For a symbolic center c and radius $R = 20$, the exported symbolic context is the word*

$$\Gamma_R(c) = (\sigma_{c-R}, \dots, \sigma_c, \dots, \sigma_{c+R})$$

of length $2R + 1$ in the return-block sequence. This is the symmetric window used in the certificate.

Definition 6.3 (Symbolic encoding map). *The exporter defines a map from recurrence data and symbolic contexts to certified symbolic realizations whenever the local data match one of the certified symbolic patterns.*

The certificate records, for every exported realization, the global target index, the local symbolic target position, the symbolic label, the two dependency offsets, the two argument labels, the certified values, and the parity term.

Definition 6.4 (Arithmetic certificate record). *Each exported realization record stores a global block anchor L . Each arithmetic record belonging to that realization has the form*

$$(t, q_t, r_a, o_a, \sigma_a, q_a, r_b, o_b, \sigma_b, q_b, \varepsilon),$$

where:

- t is the local target offset relative to the block anchor L ;
- q_t is the certified value at the target;
- r_a, r_b are the relative offsets of the two recursive calls;
- o_a, o_b are auxiliary offsets recorded by the exporter;
- σ_a, σ_b are the symbolic labels of the two argument positions;
- q_a, q_b are the certified values at the two recursive arguments;
- $\varepsilon \in \{-1, 1\}$ is the certified perturbation term.

The corresponding global target index is recoverable from the exported block anchor L and the local target offset t , namely

$$n = L + t.$$

Thus the parity term ε is not merely a symbolic label: the checker verifies it against the actual integer parity of the corresponding global target index.

The intended semantics of the record is

$$q_t = q_a + q_b + \varepsilon, \quad t + r_a < t, \quad t + r_b < t,$$

with the parity condition

$$\varepsilon = (-1)^{L+t}.$$

Definition 6.5 (Word certificate). *The symbolic word certificate consists of four finite word systems*

$$S, T, U, V$$

with sizes

$$|S| = 53, \quad |T| = 46, \quad |U| = 38, \quad |V| = 28.$$

Each T -word is a word over the S -alphabet, each U -word is a word over the T -alphabet, and each V -word is a word over the U -alphabet.

The stable core consists of

$$V_0, \dots, V_{27}.$$

The corresponding S -level alphabet is

$$\{S_0, \dots, S_{52}\}.$$

Definition 6.6 (Extension record). *An extension record has the form*

$$E(a, b, c),$$

where a, b, c are context identifiers. If

$$\Gamma_a = (x_0, x_1, \dots, x_{2R}) \quad \text{and} \quad \Gamma_b = (y_0, y_1, \dots, y_{2R})$$

are the corresponding certified symbolic contexts, then the record is admissible only if

$$x_i = y_{i-1} \quad (1 \leq i \leq 2R).$$

In that case c is the identifier of the shifted context

$$\Gamma_c = (x_1, x_2, \dots, x_{2R}, y_{2R}).$$

Thus an extension record is a certified radius- R shift transition between symbolic contexts.

Definition 6.7 (Admissible certified state). *A certified symbolic state is admissible if it occurs as a declared context in the exported context table and all symbols appearing in it belong to the declared finite word alphabet.*

A certified successor transition is admissible if it is represented by an extension record

$$E(a, b, c)$$

satisfying the suffix-prefix overlap condition

$$\Gamma_a[1, \dots, 2R] = \Gamma_b[0, \dots, 2R - 1],$$

with c equal to the corresponding shifted context

$$\Gamma_c = (\Gamma_a[1], \dots, \Gamma_a[2R], \Gamma_b[2R]).$$

Definition 6.8 (Certified symbolic transition relation). *The certified symbolic transition relation is the finite relation on declared symbolic contexts whose edges are the admissible extension records $E(a, b, c)$ contained in the certificate.*

Lemma 6.9 (Real-to-certified transition simulation). *Let a real local configuration be encoded by a certified symbolic context Γ_a . Suppose that the defining recurrence determines the next local symbolic context by shifting the radius- R window one step forward. Then the resulting successor context is represented by an extension record*

$$E(a, b, c)$$

in the certified transition relation, and the encoded successor context is precisely Γ_c .

Equivalently, whenever the real recurrence produces a successor of an encoded local configuration, the symbolic encoding of that successor agrees with the certified radius- R shift transition recorded in the certificate.

Proof. This is the finite simulation property checked by the combination of the extension records and the faithfulness checker. An extension record

$$E(a, b, c)$$

is admissible exactly when the contexts Γ_a and Γ_b satisfy the suffix-prefix overlap condition

$$\Gamma_a[1, \dots, 2R] = \Gamma_b[0, \dots, 2R - 1],$$

and Γ_c is the shifted context

$$\Gamma_c = (\Gamma_a[1], \dots, \Gamma_a[2R], \Gamma_b[2R]).$$

The faithfulness checker independently recomputes the trace from the original recurrence, reconstructs the symbolic contexts and the extension relation, and verifies that the exported extension records agree exactly with this recomputation. For each reachable declared admissible context a , the checker computes the required successor context c_{req} from the certified radius- R shift rule independently of the exported edge list. It then checks that the certificate contains an extension record

$$E(a, b, c_{\text{req}})$$

with the required suffix-prefix overlap. Thus

$$\text{extension_missing_bad} = 0$$

means that, within the reachable certified domain, every independently computed required successor shift is present as an extension record in the certificate. The additional check

$$\text{extension_bad} = 0$$

means that every declared extension record satisfies the required suffix-prefix overlap and shifted-context identity.

Therefore, on the declared reachable admissible states, the real successor operation after symbolic encoding agrees with the certified transition relation. $\square$

Definition 6.10 (Certified history invariant). *For $n \geq N$, let $\mathcal{I}(n)$ denote the assertion that the real recurrence history up to index n , starting from the verified base trace, is encoded by a path in the certified symbolic transition relation. Equivalently, every local symbolic context needed to evaluate the recurrence up to index n is represented by an admissible certified context, and every shift from one such context to the next is represented by an extension record in the certificate.*

Lemma 6.11 (Propagation of the certified history invariant). *If $\mathcal{I}(n)$ holds and the values*

$$Q(1), \dots, Q(n)$$

are defined, then the real successor context at index $n+1$, obtained from the defining recurrence, is represented by a certified successor context. In particular,

$$\mathcal{I}(n) \implies \mathcal{I}(n+1)$$

whenever the recurrence step from n to $n+1$ is defined.

Proof. Assume that $\mathcal{I}(n)$ holds. Then the real local context at the current index is encoded by an admissible certified context. By Lemma 6.9, the real successor context produced by shifting the radius- R window agrees with a certified extension record

$$E(a, b, c)$$

in the exported transition relation. By Theorem 6.13, this certified successor remains inside the finite certified symbolic universe after the verified cycle factorization. Hence the encoded real history extends by one certified transition step, and $\mathcal{I}(n+1)$ holds. $\square$

Theorem 6.12 (Anchored realization of the verified trace). *Every real recursive configuration occurring in the verified base trace is either directly exported as a certified symbolic realization or is contained in the anchored symbolic core from which the certified transition relation starts.*

Object	Meaning	Checked by	Used for
S, T, U, V	symbolic block systems	composition and factorization checkers	cycle closure
Γ_R	radius- R symbolic contexts	faithfulness and extension checks	transition propagation
$E(a, b, c)$	certified context shift	faithfulness checker	successor simulation
realizations	symbolic recurrence instances	faithfulness checker	link to recurrence data
arithmetic records	local recursive identities	induction checker	backward induction

Table 1: Roles of the main certificate objects in the proof.

Proof. The trace-generation program computes the recursion directly up to

$$N = 5 \cdot 10^7$$

and checks at each step that both recursive arguments are positive. The exporter records local recursive configurations from this verified trace and assigns the corresponding symbolic labels whenever the configuration matches one of the certified symbolic patterns.

The stable-prefix certificate identifies the stable core

$$V_0, \dots, V_{27}$$

with the corresponding certified S -patterns. Hence the verified trace supplies an explicit anchor for the finite symbolic transition system. $\square$

Theorem 6.13 (Certified closure of symbolic successors). *Every admissible symbolic successor of a certified symbolic state is again represented in the certified symbolic universe after one complete cycle and factorization into certified S -words.*

Proof. This is the finite statement checked by

`coverage_anchor_checker.c.`

The checker verifies the word counts

$$|S| = 53, \quad |T| = 46, \quad |U| = 38, \quad |V| = 28,$$

the symbol bounds, the stable-prefix identification

$$V_i \equiv S_i \quad (0 \leq i < 28),$$

the full-cycle expansion

$$V \rightarrow U \rightarrow T \rightarrow S,$$

and complete factorization into certified S -words.

The verified output reports

`symbol_bounds = OK,      stable_prefix = OK,      composition_factorization = OK,`

together with

$$\text{symbol_bad} = 0, \quad \text{factor_bad} = 0, \quad \text{factor_count_mismatch} = 0.$$

Therefore no admissible certified symbolic successor leaves the finite certified symbolic universe. $\square$

Theorem 6.14 (Faithfulness of the exported certificate). *The exported words, contexts, extensions, realizations, dependency offsets, recurrence values, and parity terms agree exactly with the data obtained by independently recomputing the verified trace and rerunning the symbolic extraction pipeline on that trace.*

Proof. This is the finite statement checked by

`faithfulness_checker.c`.

The checker recomputes

$$Q(1), Q(2), \dots, Q(N)$$

directly from the defining perturbed Hofstadter recurrence. From this recomputed trace it reconstructs the derived T -, S -, and $C7$ -arrays, the return-word dictionary, the radius- R context table, the extension relation, and the realization records.

It then parses the exported certificate and compares every declared word, context, extension, and realization record with the independently recomputed data. For every arithmetic record it recomputes the target index, the two recursive argument indices, the relative offsets, the argument symbols, the values

$$q_t, \quad q_a, \quad q_b,$$

and the parity term

$$\varepsilon = (-1)^n$$

from the original recurrence trace, and checks exact equality with the exported record.

The verified output reports

$$\begin{aligned} \text{malformed_bad} &= 0, & \text{word_bad} &= 0, & \text{context_bad} &= 0, \\ \text{extension_bad} &= 0, & \text{extension_missing_bad} &= 0, & \text{realization_bad} &= 0, \\ \text{record_bad} &= 0, & \text{header_bad} &= 0. \end{aligned}$$

Hence the exported certificate is faithful to the independently recomputed trace and symbolic extraction pipeline. The use of this finite data beyond the verified trace is justified separately by the symbolic closure and finite-state induction checks. $\square$

Corollary 6.15 (Certified global symbolic coverage). *By Theorems 6.12, 6.13, 6.14, and Lemma 6.9, every recursive configuration generated from the anchored certified core by the defining recurrence and encoded through the certified symbolic map is represented by a certified symbolic realization.*

Proof. By Theorem 6.12, the verified base trace anchors the symbolic system in actual recursive configurations of the sequence. By Lemma 6.9, whenever an encoded real configuration produces a successor under the defining recurrence, the symbolic encoding of that successor agrees with a certified extension record.

By Theorem 6.13, every admissible symbolic successor of a certified symbolic state returns, after one full cycle and factorization, to certified S -words. By Theorem 6.14, the exported transition and realization records agree with an independent recomputation from the original recurrence on the certified domain.

Thus repeated recurrence-generated propagation from the anchored core cannot leave the finite certified symbolic universe. Hence every configuration generated from the anchored core and represented by the certified transition relation is represented by a certified symbolic realization. $\square$

7 Coverage and Anchoring

The symbolic certificate must be connected to the actual recursive sequence. This connection is supplied by the directly verified initial trace, the positivity bound for certified dependencies, and the global coverage statement derived from the certified transition system.

Throughout this section we write

$$N = 5 \cdot 10^7$$

for the length of the directly verified initial trace.

Lemma 7.1 (Verified base range). *The trace-generation program computed the recursion directly up to N and verified at each step that both recursive arguments were positive. Within this verified range, the exported symbolic and arithmetic certificates are anchored to actual recursive configurations of the sequence.*

Proof. The trace-generation program computes the values

$$Q(1), Q(2), \dots, Q(N)$$

successively from the defining recursion. At each step it checks that both recursive arguments

$$n - Q(n - 1), \quad n - Q(n - 2)$$

are positive and therefore refer to previously computed values. The same verified trace is used to export the symbolic certificates and the arithmetic recurrence certificate checked by the independent verification programs. $\square$

Lemma 7.2 (Positivity anchoring). *Every certified recursive dependency with global target index $n > N$ corresponds to a positive global index.*

Proof. The finite trace verifies the recursion directly up to N . For certified recursive calls beyond this range, the induction checker reports

$$\text{min_rel_seen} = -2441147.$$

Hence every certified recursive argument from a certified target index $n > N$ is bounded below by

$$n + \text{min_rel_seen} \geq N + 1 - 2441147 = 47558854 > 0.$$

Thus every certified recursive dependency with target index $n > N$ refers to a positive earlier index. $\square$

Lemma 7.3 (Symbolic coverage and anchoring). *The stable core $V_0, \dots, V_{27}$ is anchored to the corresponding S -patterns, and every recursive symbolic descendant generated by one complete cycle factors into certified S -words.*

Proof. This is the finite statement checked by

$$\text{coverage_anchor_checker.c.}$$

The checker verifies

$$\text{symbol_bounds} = \text{OK}, \quad \text{stable_prefix} = \text{OK}, \quad \text{composition_factorization} = \text{OK}.$$

In particular, it checks that

$$V_i \equiv S_i \quad (0 \leq i < 28),$$

and that every V_i , $0 \leq i < 28$, expands through the full symbolic cycle

$$V \rightarrow U \rightarrow T \rightarrow S$$

to a raw word over the S -alphabet which factors completely into certified S -words. The checker reports

$$\text{symbol_bad} = 0, \quad \text{factor_bad} = 0, \quad \text{factor_count_mismatch} = 0.$$

Thus symbolic propagation from the anchored stable core cannot generate a local symbolic configuration outside the certified symbolic system. $\square$

Lemma 7.4 (Global symbolic coverage). *Let $n > N$. If all values*

$$Q(1), \dots, Q(n-1)$$

are defined and the local recurrence history up to n is obtained by successive certified transition propagation from the anchored base trace, then the local recursive configuration needed to evaluate $Q(n)$ is represented by a certified symbolic realization.

Proof. This follows from Corollary 6.15 together with the anchoring of the verified base range. $\square$

Although the stable prefix has length 28, the factorization may use S -words outside this prefix. These are nevertheless certified S -level words and are part of the finite symbolic universe. The stable-prefix condition anchors the cyclic return, while factorization closure is taken with respect to the full S -system

$$\{S_0, \dots, S_{52}\}.$$

8 Main Theorem

We now complete the proof of global well-definedness.

Recall that the perturbed Hofstadter recursion is

$$Q(1) = Q(2) = 1,$$

and for $n \geq 3$,

$$Q(n) = Q(n - Q(n-1)) + Q(n - Q(n-2)) + (-1)^n.$$

The sequence is well-defined if, for every $n \geq 3$, both recursive arguments

$$n - Q(n-1), \quad n - Q(n-2)$$

are positive and hence refer to previously defined values.

The finite verification developed above gives a certified symbolic induction mechanism. We now show that this mechanism rules out the possibility of a first undefined value.

Theorem 8.1 (Well-definedness). *The perturbed Hofstadter recursion*

$$Q(n) = Q(n - Q(n-1)) + Q(n - Q(n-2)) + (-1)^n, \quad Q(1) = Q(2) = 1,$$

is well-defined for all $n \geq 1$.

Proof. Assume, for contradiction, that the recursion is not well-defined for all $n \geq 1$. Then there exists a smallest index

$$n_0 \geq 3$$

at which the recurrence fails.

If $n_0 \leq N$, this contradicts Lemma 7.1. Thus $n_0 > N$.

By minimality of n_0 , all values

$$Q(1), Q(2), \dots, Q(n_0 - 1)$$

are already defined. Therefore, the only possible failure at n_0 is that at least one of the recursive arguments

$$n_0 - Q(n_0 - 1), \quad n_0 - Q(n_0 - 2)$$

is not a valid earlier index.

By Corollary 6.15, the local recursive configuration needed to evaluate $Q(n_0)$ is represented by a certified symbolic realization. The use of the corollary here is justified by induction on the target index. The verified base trace anchors the invariant $\mathcal{I}(N)$. Lemma 6.11 shows that one real recurrence step corresponds to one certified transition step, and Theorem 6.13 shows that the certified successor remains in the finite certified universe. Hence the actual recurrence history up to index $n_0 - 1$ lies in the certified transition propagation from the anchored base trace, and the local configuration needed to test the recurrence at n_0 is represented by a certified symbolic realization.

Hence the two recursive dependencies at n_0 are certified dependencies checked by the induction certificate. By Lemma 7.2, these certified dependencies correspond to positive global indices.

By the certified closure of symbolic successors and the composition closure theorem, recursive propagation of this certified realization remains inside the finite symbolic universe. Hence recursive propagation does not create any new symbolic object outside the certified system.

By arithmetic and parity consistency, every certified recurrence identity satisfies

$$q_t = q_a + q_b + \varepsilon, \quad \varepsilon = (-1)^n,$$

for the corresponding global target index n . Thus the symbolic recurrence equations agree exactly with the defining perturbed Hofstadter recursion.

By strict backwardness, every certified recursive dependency is directed to a strictly earlier position. In the notation of the certificate, the relative offsets r_a and r_b of the two recursive calls satisfy

$$r_a < 0, \quad r_b < 0.$$

Consequently, both recursive calls associated with the target position refer to positive smaller indices.

Therefore, at the supposed first failing index n_0 , every recursive dependency appearing in the certified realization points to an index strictly smaller than n_0 . By the minimality of n_0 , all such values are already defined.

It follows that both recursive terms

$$Q(n_0 - Q(n_0 - 1)), \quad Q(n_0 - Q(n_0 - 2))$$

are defined, and arithmetic consistency then determines $Q(n_0)$. This contradicts the assumption that n_0 is the first index at which the recursion fails.

Hence no such n_0 exists, and the recursion is well-defined for all $n \geq 1$. $\square$

8.1 Logical dependencies of the proof

The proof above uses exactly the following verified ingredients:

1. *Finite symbolic closure*: recursive symbolic expansion stays inside the certified finite symbolic system.
2. *Cycle factorization*: every four-step composed expansion of the stable V -core factors into certified S -words.
3. *Arithmetic and parity correctness*: every certified recurrence identity satisfies

$$q_t = q_a + q_b + \varepsilon$$

with

$$\varepsilon = (-1)^n$$

for the corresponding global target index, and therefore matches the defining recurrence exactly.

4. *Strict backwardness*: every recursive call in the certificate points to a strictly earlier index.
5. *Inductive minimality*: if a first failure existed, all earlier indices would already be defined.

The computer-assisted part of the proof is therefore confined to finite, explicitly checkable certificates. The final passage from these certificates to global well-definedness is the ordinary minimal-counterexample argument given above.

8.2 Certificate summary

The final verification data used in the proof are:

$$|S| = 53, \quad |T| = 46, \quad |U| = 38, \quad |V| = 28.$$

The coverage and anchoring verification reports:

$$\begin{aligned} \text{symbol_bounds} &= \text{OK}, & \text{stable_prefix} &= \text{OK}, \\ \text{composition_factorization} &= \text{OK}, & \text{checked_core_words} &= 28, \\ \text{symbol_bad} &= 0, & \text{factor_bad} &= 0, \\ \text{factor_count_mismatch} &= 0. \end{aligned}$$

The backward-induction checker processed

$$\text{realizations_checked} = 4,735,941, \quad \text{arithmetic_records_checked} = 49,999,571.$$

The backward induction verification reports:

$$\begin{aligned} \text{min_rel_seen} &= -2441147, & \text{max_rel_seen} &= -10, \\ \text{min_global_target_seen} &= 290, & \text{max_global_target_seen} &= 49999860, \\ \text{arithmetic_bad} &= 0, & \text{non_backward_bad} &= 0, \\ \text{parity_bad} &= 0, & \text{malformed_bad} &= 0, \\ \text{header_bad} &= 0. \end{aligned}$$

These global target indices refer to certified recurrence records in the exported finite certificate; the separate positivity anchoring lemma is used for certified dependencies whose target index lies beyond the directly verified base range. Thus every certified recursive call is strictly backward, every certified arithmetic identity is correct, every certified parity term is consistent with its global target index, and the finite symbolic system is closed under the verified recursive composition cycle.

8.3 Conclusion

The perturbed Hofstadter recursion admits a finite certified symbolic induction system. This system is closed under recursive composition, arithmetically consistent, and strictly backward. Therefore recursive evaluation can never encounter a first undefined index.

This completes the proof of the main theorem.

9 Computational Verification

The proof relies on a collection of independent verification programs. Each program checks a finite statement about the symbolic system or about the exported recurrence certificate. The programs do not use randomness, statistical inference, or heuristic acceptance criteria. Every accepted claim is reduced to an explicit finite computation.

The main verification programs are listed in Table 2.

Program	Purpose
<code>cycle_composition_checker.c</code>	symbol closure verification
<code>cycle_composition_pattern_checker.c</code>	diagnostic pattern realization check
<code>cycle_composition_factor_checker.c</code>	factorization certification
<code>coverage_anchor_checker.c</code>	stable-prefix, symbolic coverage, and anchored cycle factorization
<code>well_definedness_induction_checker.c</code>	arithmetic correctness, parity consistency, and strict backward induction
<code>faithfulness_checker.c</code>	recomputation of the trace and verification of the exported transition and realization data

Table 2: Main verification programs used in the proof.

9.1 Role of the computational component

The computational component serves two distinct purposes.

First, it constructs finite symbolic data from verified traces of the recursion. This includes the symbolic word systems

$$S, \quad T, \quad U, \quad V,$$

their transition sequences, and their factorization structure.

Second, it verifies the finite certificates used in the proof. These verification steps are logically separate from the extraction step. Once the finite certificates have been exported, the checkers operate only on explicit machine-readable data.

Thus the proof does not depend on visual pattern recognition or informal interpretation of the trace. It depends on finite certificates and on deterministic programs that verify them.

9.2 Certificate format

The certificate format and the realization map are specified in Section 6. We summarize here only the parts of the certificate that are checked computationally.

The certificate consists of the following finite data:

1. finite symbolic word systems

$$S, \quad T, \quad U, \quad V,$$

together with their declared sizes;

2. a finite list of contexts representing admissible local symbolic configurations;

3. a finite extension relation describing the symbolic successor operation generated from the anchored core;
4. a finite list of symbolic realizations connecting symbolic states to local recurrence instances;
5. arithmetic records of the form

$$(t, q_t, r_a, o_a, \sigma_a, q_a, r_b, o_b, \sigma_b, q_b, \varepsilon),$$

where t is the local target position, q_t is the certified target value, r_a, r_b are the relative offsets of the two recursive calls, σ_a, σ_b are symbolic labels of the argument positions, q_a, q_b are the certified argument values, and ε is the certified perturbation term.

For each arithmetic record the induction checker verifies

$$q_t = q_a + q_b + \varepsilon,$$

strict backwardness

$$r_a < 0, \quad r_b < 0,$$

and parity consistency

$$\varepsilon = (-1)^n$$

for the corresponding global target index n .

The faithfulness checker independently recomputes

$$Q(1), Q(2), \dots, Q(N)$$

from the original perturbed Hofstadter recurrence. From this recomputed trace it reconstructs the derived T -, S -, and $C7$ -arrays, the return-word dictionary, the radius- R context table, the extension relation, and all realization records.

It then parses the exported certificate and checks exact equality between the recomputed data and the declared words, contexts, extensions, realizations, dependency offsets, recurrence values, and parity terms. Thus the certificate is not merely a collection of output logs: it is a finite machine-readable object whose entries are independently checked against a recomputation from the defining recurrence.

9.3 Symbol closure checker

The program

`cycle_composition_checker.c`

checks that the four-step composition

$$V \rightarrow U \rightarrow T \rightarrow S$$

does not leave the certified symbolic alphabet.

For each stable word

$$V_i, \quad 0 \leq i < 28,$$

the checker expands V_i through the certified word systems U, T, S and verifies that every resulting symbol lies in

$$\{0, \dots, 52\}.$$

The verified result is:

$$\text{checked_words} = 28,$$

$$\text{ok_words} = 28,$$

$$\text{total_bad_final_symbols} = 0.$$

This proves that symbolic recursive composition introduces no forbidden symbols.

9.4 Pattern realization checker

The program

`cycle_composition_pattern_checker.c`

checks whether the expanded four-step compositions coincide directly with single certified S -words.

This is a diagnostic test for a stronger closure property which is not required by the proof and is not expected to hold. The checker confirms that the expanded compositions remain inside the S -alphabet, but reports that only one of the 28 expanded words is itself a single S -word:

`matched_S_words = 1,      unmatched = 27,      symbol_bad_total = 0.`

Thus the correct closure notion is not equality with one certified S -word, but complete factorization into certified S -words. This is the property verified by

`cycle_composition_factor_checker.c.`

9.5 Factorization checker

The program

`cycle_composition_factor_checker.c`

verifies the key closure statement used in the proof.

For every

$$0 \leq i < 28,$$

it expands V_i through the cycle

$$V \rightarrow U \rightarrow T \rightarrow S$$

and then factors the resulting word as a concatenation of certified S -words.

The verified output is:

`checked_words = 28,`

`factored = 28,`

`failed = 0,`

`symbol_bad_total = 0.`

Therefore every stable V -word returns, after one full symbolic cycle, to a concatenation of certified S -words.

This is the computational certificate for recursive symbolic closure.

9.6 Coverage and anchoring checker

The program

`coverage_anchor_checker.c`

verifies the symbolic bridge between the stable core and the certified S -system.

It checks:

1. that the word systems S, T, U, V have sizes

`53,   46,   38,   28;`

2. that all symbolic references are within bounds;

3. that the stable prefix satisfies

$$V_i \equiv S_i \quad (0 \leq i < 28);$$

4. that every V_i , $0 \leq i < 28$, expands through

$$V \rightarrow U \rightarrow T \rightarrow S$$

to a raw word over the S -alphabet;

5. that every such raw word factors completely into certified S -words.

The verified output was:

`symbol_bounds = OK,      stable_prefix = OK,      composition_factorization = OK.`

The checker also reports

`symbol_bad = 0,      factor_bad = 0,      factor_count_mismatch = 0.`

The coverage-anchor checker repeats the relevant cycle-factorization test in an anchored form and additionally verifies the stable-prefix identification $V_i \equiv S_i$ for $0 \leq i < 28$.

9.7 Backward induction checker

The program

`well_definedness_induction_checker.c`

checks the exported recurrence certificate.

It verifies:

1. that every arithmetic recurrence identity is correct;
2. that every perturbation term satisfies the parity condition

$$\varepsilon = (-1)^n$$

for the corresponding global target index;

3. that every recursive dependency is strictly backward;
4. that the number of checked realizations matches the certificate header;
5. that no malformed arithmetic records occur.

The checker processed:

4,735,941

recursive realizations and

49,999,571

arithmetic records.

The final output was:

`arithmetic_bad = 0,
non_backward_bad = 0,
parity_bad = 0,
malformed_bad = 0.`

It also reported:

`min_rel_seen = -2441147,      max_rel_seen = -10.`

Thus every certified recursive call is strictly backward.

9.8 Faithfulness checker

The program `faithfulness_checker.c` independently recomputes the verified trace from the original recurrence, reconstructs the symbolic extraction pipeline, and checks that the exported certificate agrees exactly with the recomputed words, contexts, extensions, realizations, dependency offsets, recurrence values, and parity terms.

The verified output was:

```
words_checked = 13,      contexts_checked = 2923,      extensions_checked = 3026,  
realizations_checked = 4,735,941,      arithmetic_records_checked = 49,999,571.
```

The checker reported

```
malformed_bad = 0,      word_bad = 0,      context_bad = 0,  
extension_bad = 0,      extension_missing_bad = 0,      realization_bad = 0,  
record_bad = 0,      header_bad = 0.
```

Thus the exported certificate agrees exactly with an independent recomputation of the verified trace and symbolic extraction pipeline.

9.9 Verification guarantees

All computations used in the proof have the following properties:

- *finite*: every program terminates after checking a finite certificate;
- *deterministic*: no random choices or probabilistic tests are used;
- *reproducible*: the checkers operate on explicit input files and produce deterministic output;
- *machine-verifiable*: each accepted statement is reduced to integer comparisons, exact word matching, exact factorization, or exact arithmetic verification.

The computational verification is therefore not a numerical experiment. It is a finite certificate check.

9.10 Trusted code base

The proof relies on a finite trusted computational base. The trusted components are the C source code of the verification programs, the C compiler and runtime used to build them, the parser for the machine-readable certificate files, and the exact integer arithmetic used in the checks.

All arithmetic comparisons in the checkers are performed on integer quantities whose ranges are bounded by the exported certificate and by the directly verified trace. The checkers reject malformed records and report header mismatches, out-of-range symbols, arithmetic mismatches, non-backward dependencies, and parity inconsistencies. The relevant failure counters are required to be zero.

The repository records the source files, build commands, representative logs, cryptographic hashes of the certificate artifacts and outputs, and download instructions for the large compressed certificate. Thus an independent verifier can rebuild the checkers, download the certificate artifact, compare hashes, and reproduce the zero-failure outputs reported in this paper.

A future proof-assistant formalization could further reduce the trusted code base by replacing the C checkers with formally verified parsers and certificate checkers.

9.11 Logical status of the programs

The programs play the role of finite proof checkers.

The mathematical proof uses their outputs only in the following forms:

1. the symbolic word systems have finite certified sizes

$$|S| = 53, \quad |T| = 46, \quad |U| = 38, \quad |V| = 28;$$

2. the four-step composition of each V_i , $0 \leq i < 28$, factors into certified S -words;
3. all final symbols in these compositions lie in

$$\{0, \dots, 52\};$$

4. every exported recurrence identity is arithmetically correct;
5. every exported recursive dependency is strictly backward;
6. the certificate records define an explicit finite symbolic transition relation from the anchored core, and this relation is closed under the verified cycle factorization.

These are finite statements. Once verified, they are used as inputs to the induction argument in the proof of the main theorem.

9.12 Reproducibility

The verification code and input data are provided in the accompanying repository [17]. The repository contains:

- the trace-generation program used to compute the verified base range;
- the exported symbolic certificates;
- the recurrence certificate;
- the composition checkers;
- the factorization checker;
- the coverage and anchoring checker;
- the backward induction checker;
- the faithfulness checker;
- and representative output logs.

The repository records the source code, representative output logs, compiler commands, cryptographic hashes, and download instructions for the large compressed certificate artifact.

A complete independent verification consists of recompiling the checkers and running them on the supplied certificate files. The expected output is exactly the zero-failure output recorded above.

9.13 Summary

The computational verification establishes the finite facts needed for the proof:

symbol closure + factorization closure + faithfulness
+arithmetic and parity correctness + strict backwardness.

Together these finite certificates imply the certified finite-state induction principle used in the proof of well-definedness.

10 Scope and Interpretation

The proof establishes the following statement:

The certified finite symbolic recursion system generated from the perturbed Hofstadter recursion is recursively closed, arithmetically consistent, and globally backward-well-founded.

The argument is therefore a certified finite-state induction proof. It does not derive a closed formula for $Q(n)$, nor does it attempt to describe the full fine-scale structure of the sequence. Instead, it proves that the recursive dependency mechanism can be represented inside a finite symbolic system that is closed under the relevant recursive operations.

10.1 What is proved

The proof establishes global well-definedness of the perturbed recursion

$$Q(n) = Q(n - Q(n - 1)) + Q(n - Q(n - 2)) + (-1)^n, \quad Q(1) = Q(2) = 1.$$

More precisely, it proves that recursive evaluation never encounters an undefined recursive argument. Equivalently, for every $n \geq 3$, the two arguments

$$n - Q(n - 1), \quad n - Q(n - 2)$$

are valid earlier indices.

The proof obtains this conclusion through a finite certified symbolic model. The model has the following essential verified properties:

1. *symbolic closure*: recursive symbolic expansion remains inside a finite certified alphabet;
2. *factorization closure*: composed symbolic expansions factor into certified symbolic words;
3. *backward arithmetic induction*: every certified recurrence identity is arithmetically correct and every recursive call is strictly backward;
4. *faithfulness*: the exported transition and realization data agree with an independent re-computation from the original recurrence;
5. *parity consistency*: every certified perturbation term is the actual value $(-1)^n$ for the corresponding global target index.

These properties are sufficient to exclude a first undefined index.

10.2 What is not claimed

The result is deliberately limited to well-definedness. We do not claim:

- an asymptotic formula for $Q(n)$;
- a proof of convergence of $Q(n)/n$;
- a precise error term or fluctuation law;
- uniqueness of symbolic realizations;
- classification of all possible symbolic encodings of the sequence;
- classification of all Hofstadter-type recursions with alternating perturbations;
- or a direct symbolic description of the classical Hofstadter Q -sequence.

In particular, the proof does not resolve the well-definedness problem for the classical unperturbed recursion

$$Q(n) = Q(n - Q(n - 1)) + Q(n - Q(n - 2)).$$

The alternating perturbation plays an essential role in the finite symbolic structure verified here. Asymptotic questions for the same perturbed recursion are addressed by Cloitre in [7]. The present paper is instead focused on a certificate-based finite-state induction proof of well-definedness.

10.3 Role of computation

The proof is computer-assisted, but the computational component is finite and certificate-based. The programs are not used to extrapolate from numerical evidence. Instead, they verify explicit finite statements:

- that the symbolic systems S, T, U, V have the stated finite word sets;
- that the stable V -prefix expands through the cycle

$$V \rightarrow U \rightarrow T \rightarrow S$$

without leaving the certified S -alphabet;

- that every such expansion factors into certified S -words;
- that every exported recurrence identity satisfies the defining arithmetic equation;
- and that every exported recursive dependency is strictly backward.

Thus the computations function as proof checkers for finite certificates. The final mathematical argument is an induction based on these verified finite facts.

10.4 Interpretation of the finite symbolic model

The finite symbolic model should be understood as a recursively closed representation of the dependency structure of the perturbed recursion.

It is not asserted that the symbolic words provide the only possible encoding of the sequence. Nor is it asserted that the symbolic model captures every combinatorial feature of $Q(n)$. Its role is narrower: it captures enough of the recursive dependency structure to prove that recursive evaluation cannot break down.

The key point is that recursive expansion does not generate an infinite sequence of new symbolic types. After one full renormalization cycle, every stable symbolic word returns to a factorization over the certified S -system. This is the finite-state mechanism underlying the proof.

10.5 Why the result is stronger than numerical verification

A direct computation of $Q(n)$ up to a large bound cannot by itself prove global well-definedness. Such a computation only verifies that no failure occurs before that bound.

The present proof uses computation differently. The finite trace is used to extract and certify a symbolic recursive system. Once this system is verified to be closed, arithmetically correct, and strictly backward, the ordinary induction argument applies beyond the computed range.

In particular, the length of the trace is not used as evidence for global behavior. The trace is used only to propose the finite state space. Exhaustiveness is supplied by the certificate: the verified transition relation maps every admissible certified state back into the same finite symbolic universe.

Thus the proof is not:

The sequence was computed very far and did not fail.

Rather, it is:

A finite recursively closed symbolic system was certified, and this system implies an induction principle excluding any first failure.

This distinction is essential.

10.6 Limitations

The main limitation is that the proof depends on the correctness of the formal certificate-generation pipeline. The independent checkers verify the exported certificates, but the connection between the original recursion and the exported symbolic model must be specified carefully and made reproducible.

For this reason, the accompanying repository is part of the mathematical record of the proof. It provides the certificate files, the checkers, and the output logs needed to reproduce the finite verification.

A future formalization in a proof assistant such as Lean or Coq would further reduce the trusted code base by encoding the finite certificates and the induction principle in a fully formal environment.

10.7 Conceptual significance

The result suggests that certain nested recursions, despite their nonlocal definitions, may admit finite recursive descriptions after suitable symbolic renormalization.

In the present case, the alternating perturbation produces enough structure to support a finite cyclic symbolic system:

$$S \rightarrow T \rightarrow U \rightarrow V \rightarrow S.$$

This cycle is not merely an empirical pattern; it is verified by finite composition and factorization checks.

The proof therefore provides evidence that finite-state and symbolic dynamical methods can be effective tools for studying selected meta-Fibonacci recursions.

10.8 Summary

The theorem proved in this paper should be read as a statement of certified well-definedness:

The perturbed Hofstadter recursion admits a finite certified symbolic induction system, and this system proves that the recursion is well-defined for all $n \geq 1$.

The proof is finite, reproducible, and based on exact symbolic and arithmetic verification. Its scope is global well-definedness, not a complete structural classification of the sequence.

11 Future Directions

The proof developed in this paper suggests several directions for further work. Some of these concern the present perturbed Hofstadter recursion, while others concern the broader use of finite symbolic methods for nested recurrences.

11.1 Formal verification

The most immediate next step is a formal verification of the proof in a proof assistant such as Lean or Coq.

The present proof already has a favorable structure for formalization: all symbolic objects are finite, all verification steps reduce to exact integer or word computations, and the final mathematical argument is a standard induction principle.

A formalization could proceed by encoding:

- the finite word systems S, T, U, V ;
- the symbolic expansion maps;
- the factorization certificates;
- the arithmetic recurrence records;
- the strict backwardness certificate;
- and the final induction argument.

Such a formalization would reduce the trusted part of the proof to the proof assistant kernel and the explicit certificate files.

11.2 Reducing the trusted code base

The current verification pipeline separates certificate generation from certificate checking. The exported certificate is checked independently for symbolic closure, factorization, faithfulness, arithmetic correctness, parity consistency, and strict backwardness. Nevertheless, the proof still relies on several verification programs and on the precise interpretation of the certificate format.

A natural next step is to reduce the trusted code base further by developing a smaller standalone checker for the final certificate. Such a checker would implement only the formal certificate specification: word expansion, context-extension validation, realization checking, arithmetic and parity verification, and strict backwardness. It would not perform pattern discovery or certificate generation.

Ideally, the repository should provide:

1. a stable machine-readable certificate format;
2. cryptographic hashes for all certificate and log files;
3. a short standalone verifier for the final certificate;
4. scripts reproducing all reported checker outputs;
5. and, ultimately, a proof-assistant formalization of the induction principle and certificate semantics.

This would make the computational part easier to audit independently and would further separate the mathematical proof from the discovery and extraction pipeline.

11.3 Asymptotic behavior

The present paper proves well-definedness only by a certificate-based finite-state induction method. A complementary analytic treatment of the asymptotic behavior of the same perturbed recursion was given by Cloitre [7], including the estimate

$$\left| Q(n)/n - \frac{1}{2} \right| = O\left(1/\sqrt{\log n}\right).$$

It remains an interesting question whether the finite symbolic certificates developed here can also be used to certify asymptotic properties.

11.4 Extension to other meta-Fibonacci recursions

Another direction is to apply the same strategy to other nested recursions. The general method would be:

1. generate a verified trace;
2. extract return words or symbolic blocks;
3. search for stable symbolic renormalization cycles;
4. certify composition closure;
5. verify strict backwardness;
6. and derive a finite induction principle.

This may be applicable to other Hofstadter-type recursions with periodic forcing terms, modified initial conditions, or additional correction terms.

An important open question is to determine which perturbations admit finite symbolic induction systems and which do not.

11.5 Symbolic renormalization

The cyclic structure

$$S \rightarrow T \rightarrow U \rightarrow V \rightarrow S$$

suggests a symbolic renormalization phenomenon.

In the present proof, the renormalization cycle is used only as a certificate of closure. A deeper analysis might explain why this cycle appears, whether it is unique, and how it relates to the arithmetic structure of the recursion.

Possible questions include:

- Is the cycle $S \rightarrow T \rightarrow U \rightarrow V \rightarrow S$ canonical?
- Are there smaller equivalent symbolic systems?
- Can the cycle be derived directly from the recurrence?
- Does the cycle encode asymptotic information?
- Are there analogous cycles for related recursions?

Understanding these questions could turn the present certificate-based proof into a more conceptual structural theory.

11.6 Recursive automata for nested recurrences

The proof can be viewed as constructing a finite recursive automaton for a nonlocal recurrence. This suggests a broader framework in which nested recursions are studied by finite automata equipped with recursive expansion rules.

Such automata would differ from ordinary finite automata because their transitions encode recursive dependencies rather than only local symbol updates. Nevertheless, once closure and backwardness are certified, they can support induction proofs.

Developing a general theory of such recursive automata could provide new tools for studying meta-Fibonacci systems.

11.7 Automated model extraction

At present, the symbolic model is extracted by a specialized pipeline. A useful future development would be an automated system for discovering finite symbolic models from recurrence data.

Such a system would:

1. compute a large verified trace;
2. identify candidate return-word decompositions;
3. test marker stability;
4. search for renormalization cycles;
5. generate candidate certificates;
6. and attempt independent verification.

This could turn the method into a general experimental and proof-assistant tool for nested recurrences.

11.8 Comparison with the classical Hofstadter sequence

The classical Hofstadter Q -sequence remains substantially harder. The present proof does not directly apply to it.

However, the contrast is informative. The alternating perturbation in the present recurrence appears to introduce enough structure to produce a finite symbolic renormalization cycle. It would be interesting to investigate whether partial symbolic models exist for the unperturbed case, and if so, why they fail to close.

Such an analysis might clarify which features make the classical Hofstadter sequence resistant to finite-state induction methods.

11.9 Conclusion

The present work shows that a nontrivial perturbed Hofstadter recursion admits a finite certified symbolic induction proof of well-definedness. The broader hope is that finite symbolic certification, recursive automata, and renormalization-style extraction may become useful tools for studying other nested recursions whose global behavior is difficult to access by direct analytic methods.

Acknowledgments

I thank Benoît Cloitre for his careful analysis of the first version of this paper and for identifying a gap in the proof. His comments were a major motivation for the revised certificate format and for the more explicit realization and coverage arguments in the present version.

ChatGPT by OpenAI was used for editorial and structural feedback. All mathematical claims, computations, certificates, and verification results remain the responsibility of the author.

References

- [1] A. Alkan. On a generalization of Hofstadter’s Q-sequence: A family of chaotic generational structures. *Complexity*, page Article 8517125, 2018.
- [2] A. Alkan and N. Fox. On Hofstadter-type sequences. *Complexity*, 2017:1–10, 2017.
- [3] A. Alkan, N. Fox, and O. O. Aybar. On Hofstadter heart sequences. *Complexity*, page Article 2614163, 2017.
- [4] M. Bodirsky and D. Král’. Locally consistent constraint satisfaction problems with binary constraints. In *Graph-Theoretic Concepts in Computer Science*, volume 3787 of *Lecture Notes in Computer Science*, pages 235–246. Springer, 2005.
- [5] J. M. Campbell and B. Cloitre. Meta-automatic sequences. *arXiv preprint*, 2026.
- [6] M. Celaya and Frank Ruskey. An undecidable nested recurrence relation. In *How the World Computes*, volume 7318 of *Lecture Notes in Computer Science*, pages 138–149. Springer, 2012.
- [7] Benoît Cloitre. On a perturbed Hofstadter Q-recursion. *arXiv preprint*, 2026.
- [8] B. Dalton, N. Fox, et al. Generalizations of Hofstadter’s Q-sequence. *Experimental Mathematics*, 20(1):55–70, 2011.
- [9] Nathan Fox. Finding linear-recurrent solutions to Hofstadter-like recurrences using symbolic computation. *arXiv preprint*, 2016.
- [10] Nathan Fox. A new approach to the Hofstadter Q-recurrence. *arXiv preprint*, 2018.
- [11] Nathan Fox. The behavior of a three-term Hofstadter-like recurrence with linear initial conditions. *arXiv preprint*, 2024.
- [12] Pavol Hell and Jaroslav Nešetřil. *Graphs and Homomorphisms*. Oxford University Press, 2004.
- [13] Douglas R. Hofstadter. *Gödel, Escher, Bach: An Eternal Golden Braid*. Basic Books, 1979.
- [14] OEIS Foundation Inc. Entry a394051 in the on-line encyclopedia of integer sequences. <https://oeis.org/A394051>, 2026. Accessed: 2026-03-31.
- [15] Douglas Lind and Brian Marcus. *An Introduction to Symbolic Dynamics and Coding*. Cambridge University Press, 1995.
- [16] Marco Mantovanelli. Dyadic self-similarity in a perturbed Hofstadter recursion. *arXiv preprint*, 2026.

- [17] Marco Mantovanelli. Verification code for “certified finite-state induction for a perturbed Hofstadter recursion”. <https://github.com/Mantovanelli11110/Certified-Finite-State-Induction-for-a-Perturbed-Hofstadter-Recursion>, 2026. Version corresponding to the paper.
- [18] Klaus Pinn. Order and chaos in Hofstadter’s Q-sequence. *Complexity*, 4(3):41–46, 1999.
- [19] Stephen M. Tanny. A well-behaved cousin of the Hofstadter sequence. *Discrete Mathematics*, 105:227–239, 1992.
- [20] P. Trojovský. On some properties of a meta-fibonacci sequence connected to Hofstadter sequence and möbius function. *Chaos, Solitons & Fractals*, 134:109719, 2020.
- [21] P. Trojovský. On some properties of the Hofstadter–Mertens function. *Complexity*, page Article 1816756, 2020.
- [22] Pavel Trojovský. On properties of Hofstadter’s Q-sequence. *Chaos, Solitons & Fractals*, 133:109644, 2020.